Workshop 2

Big Data Process

Topic: Car Price Prediction Using Machine Learning

Module: Big Data, 6CS030 | Herald College Kathmandu

Submitted: March 16, 2026

Members

Name	ID Number	Group
Narayan Lohani	2408869	L6CG15
Shreya Gurung	2418794	L6CG15
Diksha Maharjan	2408235	L6CG15

Table of Contents

1. Introduction 2
2. Step 1: Data Source & Automation 2
   - 2.1. Data Source 2
   - 2.2. Automation Strategy 2
3. Step 2: Data Exploration & Transformation 3
   - 3.1. Initial Dataset Overview 3
   - 3.2. Handling Missing Values 5
   - 3.3. Categorical Feature Distribution 6
   - 3.4. Feature Engineering 6
4. Step 3: Generated Insights 7
   - 4.1. Model Comparison Summary 7
5. Step 4: Deployment Strategy 8

Introduction

This report details a Big Data pipeline designed to predict the selling price of used cars based on vehicle characteristics such as brand, engine size, mileage, fuel type, transmission, and condition. Accurate price prediction benefits both buyers and sellers by establishing fair market values, reducing negotiation friction, and uncovering the key factors that drive depreciation.

The dataset, sourced from Kaggle's Car Price Prediction Dataset, contains 2,500 records across 10 attributes. The pipeline covers automated data ingestion, exploratory analysis, feature engineering, multi-model training with cross-validation, hyperparameter tuning, and final model evaluation.

Step 1: Data Source & Automation

Data Source

The dataset is sourced from Kaggle and downloaded programmatically using the kagglehub library. It includes the following columns:

- Car ID: Unique vehicle identifier
- Brand: Manufacturer (Tesla, BMW, Audi, Ford, Toyota, Honda, Mercedes)
- Year: Model year (ranging 2000–2023)
- Engine Size: Displacement in litres
- Fuel Type: Petrol, Diesel, Electric, or Hybrid
- Transmission: Manual or Automatic
- Mileage: Total distance driven (km)
- Condition: New, Like New, or Used
- Model: Specific model name (28 unique values)
- Price: Target variable in USD

Automation Strategy

Data acquisition is automated using kagglehub, which handles authentication and caching. The pipeline is designed to be rerun on any new dataset version without manual intervention:

- Download Strategy: kagglehub.dataset_download() fetches the dataset with builtin Colab caching for faster repeated runs.
- Data Flow: The raw CSV is loaded into a pandas DataFrame, cleaned, transformed, and then passed through a scikit learn pipeline for preprocessing and model training.

```
import kagglehub, os

path = kagglehub.dataset_download("nalisha/car-price-prediction-dataset")
csv_file = [f for f in os.listdir(path) if f.endswith(".csv")][0]
df = pd.read_csv(os.path.join(path, csv_file))

print(f"Dataset loaded: {df.shape[0]:,} rows x {df.shape[1]} columns")
```

```
Dataset loaded: 2,500 rows x 10 columns
  File: car_price_prediction_with_missing.csv
```

Step 2: Data Exploration & Transformation

Initial Dataset Overview

The raw dataset was inspected for shape, duplicates, memory usage, and null distribution before any cleaning was performed.

```
print(f"Rows       : {df.shape[0]:,}")
print(f"Columns    : {df.shape[1]}")
print(f"Duplicates : {df.duplicated().sum()}")
print(f"Memory     : {df.memory_usage(deep=True).sum() / 1024:.1f} KB")
```

```
Rows       : 2,500
Columns    : 10
Duplicates : 249
Memory     : 738.9 KB
```

	Data Type	Non-Null	Null Count	Null %	Unique Values
Car ID	float64	2250	250	10.0	2250
Brand	object	2250	250	10.0	7
Year	float64	2250	250	10.0	24
Engine Size	float64	2250	250	10.0	51
Fuel Type	object	2250	250	10.0	4
Transmission	object	2250	250	10.0	2
Mileage	float64	2250	250	10.0	2242
Condition	object	2250	250	10.0	3
Price	float64	2250	250	10.0	2249
Model	object	2250	250	10.0	28

```
=====================================================

  Rows        : 2,500

  Columns     : 10

  Duplicates  : 249

  Memory      : 738.9 KB

=====================================================
```

	Car ID	Brand	Year	Engine Size	Fuel Type	Transmission	Mileage	Condition	Price	Model
0	1.0	Tesla	2016.0	2.3	Petrol	Manual	114832.0	New	26613.92	Model X
1	2.0	BMW	2018.0	4.4	Electric	Manual	143190.0	Used	14679.61	5 Series
2	3.0	Audi	2013.0	4.5	Electric	Manual	181601.0	New	44402.61	A4
3	4.0	Tesla	2011.0	4.1	Diesel	Automatic	68682.0	New	86374.33	Model Y
4	5.0	Ford	2009.0	2.6	Diesel	Manual	223009.0	Like New	73577.10	Mustang
5	6.0	Audi	2019.0	2.4	Diesel	Automatic	246553.0	Like New	88969.76	Q7
6	7.0	Audi	2020.0	4.0	Electric	Automatic	135486.0	Used	63498.75	Q5
7	8.0	Tesla	2017.0	5.3	Hybrid	Automatic	83030.0	New	17381.19	Model Y
8	9.0	Honda	2023.0	5.7	Electric	Manual	120360.0	Like New	15905.62	Civic
9	10.0	Ford	2010.0	1.5	Electric	Automatic	135009.0	Like New	9560.22	Explorer

```
── Numeric Summary ──
```

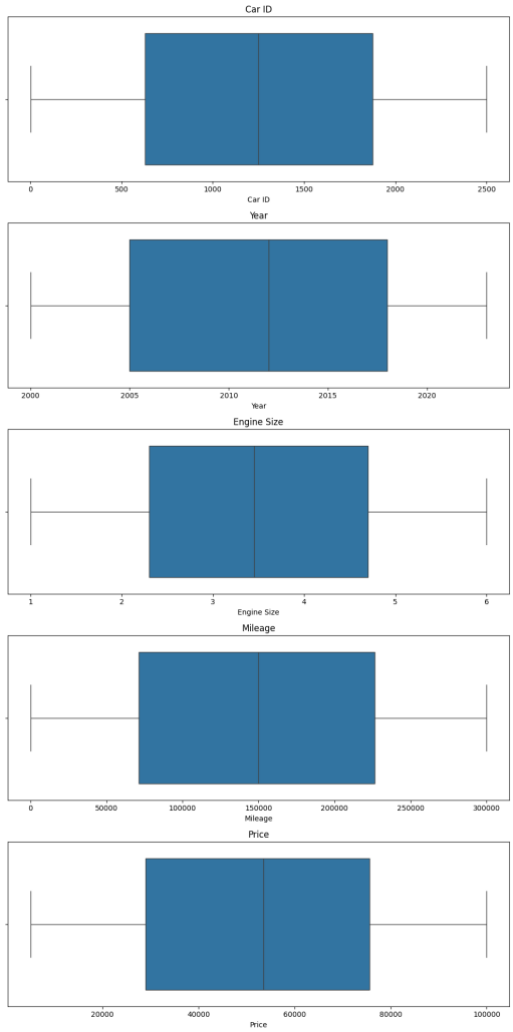
	count	mean	std	min	25%	50%	75%	max
Car ID	2250.000000	1251.243556	725.637358	1.000000	627.250000	1249.500000	1875.250000	2500.000000
Year	2250.000000	2011.577778	6.980468	2000.000000	2005.000000	2012.000000	2018.000000	2023.000000
Engine Size	2250.000000	3.485467	1.427690	1.000000	2.300000	3.450000	4.700000	6.000000
Mileage	2250.000000	150236.178222	88150.746556	15.000000	71178.500000	149762.000000	226299.000000	299967.000000
Price	2250.000000	52506.874391	27232.417079	5011.270000	28985.052500	53485.240000	75560.340000	99982.590000

```
── Categorical Summary ──
```

	count	unique	top	freq
Brand	2250	7	Toyota	346
Fuel Type	2250	4	Diesel	587
Transmission	2250	2	Manual	1184
Condition	2250	3	Used	769
Model	2250	28	Corolla	97

Handling Missing Values

All 10 columns had exactly 10% missing values (250 rows). Box plots of numeric columns confirmed that the distributions were approximately symmetric with medians near the centre of the interquartile range consistent with data missing at random (MAR). Given that the dataset exceeds the minimum 1,000 row requirement even after removal, all null rows were dropped rather than imputed.



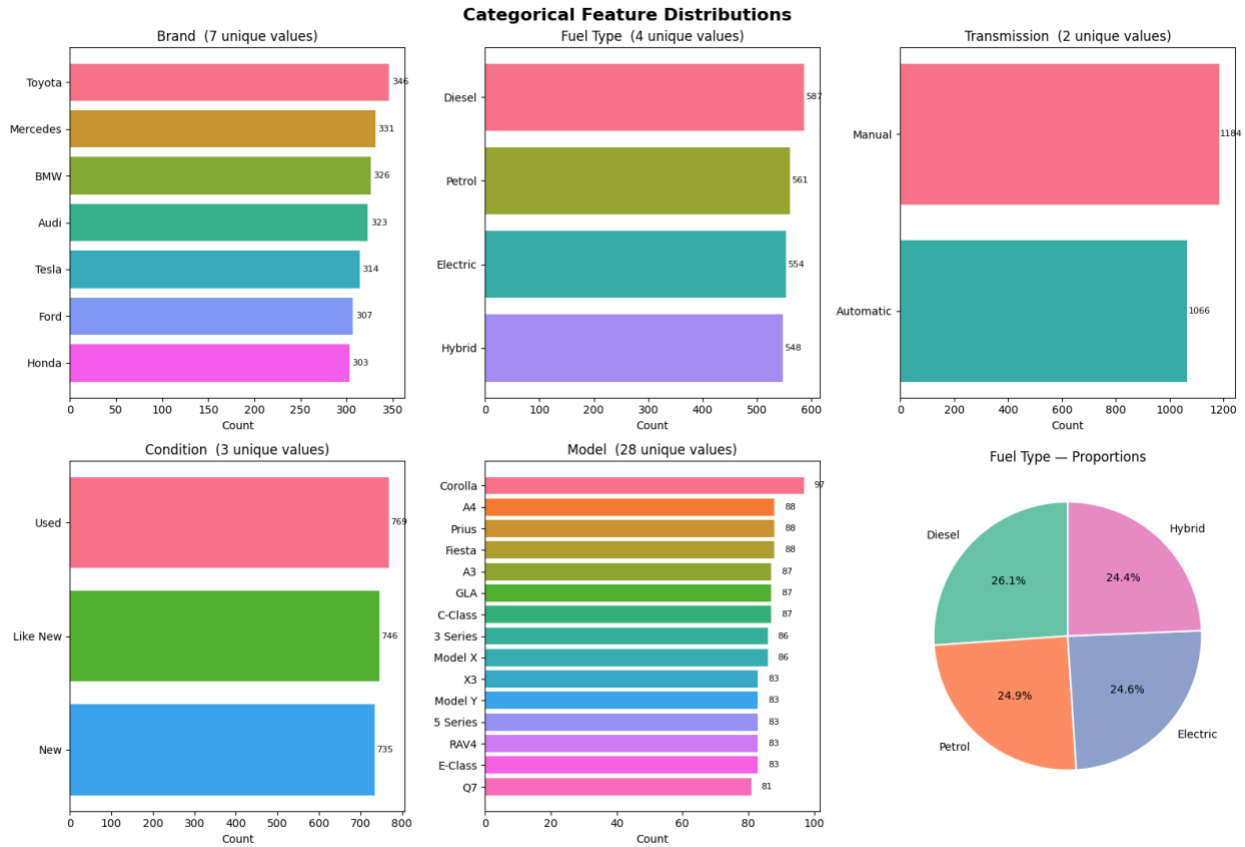

```
# Drop rows with any null values
print(f"Original shape: {df.shape}")
df.dropna(inplace=True)
print(f"Shape after dropping nulls: {df.shape}")

# Remove duplicates (resolved after null drop)
df.drop_duplicates(inplace=True)
print(f"Final shape: {df.shape}")
```

```
Original shape:          (2500, 10)
Shape after dropping nulls: (2250, 10)
Final shape:             (2250, 10)
```

Categorical Feature Distribution

After cleaning, categorical features were visualised with horizontal bar charts and a pie chart. Key observations from the distributions:

- Brand: 7 manufacturers with Toyota being the most frequent (346 records).
- Fuel Type: Diesel leads at 587 records (~26.1%), followed closely by Petrol, Electric, and Hybrid in near-equal proportions per the pie chart.
- Transmission: Manual (1,184) slightly outnumbers Automatic (1,066).
- Condition: Used (769) is the most common, followed by Like New and New.
- Model: 28 unique models; Corolla is the top model with 97 records.



Feature Engineering

Four derived features were created to improve model signal quality:

```
current_year = 2024
df["car_age"]              = current_year - df["Year"]
df["mileage_per_year"]     = df["Mileage"] / (df["car_age"] + 1)
df["engine_age_interaction"] = df["Engine Size"] * df["car_age"]
df["price_per_engine"]     = df["Price"] / (df["Engine Size"] + 0.1)

# Cap price outliers at 1st and 99th percentile
price_upper = df["Price"].quantile(0.99)
```

```
price_lower = df["Price"].quantile(0.01)
df["Price"] = np.clip(df["Price"], price_lower, price_upper)

# Skewness check - log transform only if |skew| > 1
# Result: skewness = -0.02 → original price used as target
```

```
Price skewness: -0.02

Using original price as target
```

The high-cardinality Model column (28 values) was handled with frequency encoding, adding both absolute count (Model_freq) and normalised frequency (Model_freq_norm) columns, then dropping the original string column. The final feature matrix contains 12 predictors.

Step 3: Generated Insights

Our analysis and model results revealed the following key insights:

1. **Model Complexity Dominates:** Linear models (Ridge, Lasso, ElasticNet) plateau at $R^2 \approx 0.69$, suggesting strong non-linear relationships between vehicle attributes and price. Tree-based ensembles capture these interactions and achieve $R^2 > 0.99$.
2. **Gradient Boosting Outperforms:** Gradient Boosting achieved the highest cross-validation R^2 of 0.9974, slightly ahead of XGBoost (0.9927) and Random Forest (0.9956). After hyperparameter tuning (learning_rate=0.05, max_depth=7, n_estimators=300), the CV score improved to 0.9981.
3. **Exceptional Test Set Accuracy:** The final tuned model achieves $R^2 = 0.9983$ on the held-out test set, with a Mean Absolute Error of only $695.71 and MAPE of 1.59% - meaning predictions are within roughly $700 of the actual price on average.
4. **Feature Engineering Adds Signal:** Derived features such as car_age and mileage_per_year encode depreciation dynamics more directly than raw Year and Mileage, improving the model's ability to separate high-value low-mileage vehicles from heavily-used ones of similar age.
5. **Price Distribution is Near-Symmetric:** With a price skewness of only −0.02, no log transformation was needed simplifying interpretation and back-transformation of predictions.

Model Comparison Summary

Model	Mean R^2	Std Dev (±2σ)
Ridge	0.6881	±0.0781
Lasso	0.6881	±0.0781
ElasticNet	0.6881	±0.0781
Random Forest	0.9956	±0.0017
Gradient Boosting	**0.9974**	**±0.0009**
XGBoost	0.9927	±0.0008

Note: Gradient Boosting (highlighted) is selected as the best model. Linear models are limited by their inability to capture feature interactions.

Step 4: Deployment Strategy

To make the car price prediction model accessible to end-users such as dealers, individual buyers, and sellers, the pipeline will be deployed as follows:

Deployment Platform: A Streamlit Web Application that allows users to input car attributes through an interactive form and instantly receive a predicted price with a confidence range. The app will be hosted on Streamlit Community Cloud for free public access.

API Layer: A lightweight FastAPI service will expose a /predict endpoint, enabling integration with third-party car marketplace websites and mobile applications via REST API calls.

Model Persistence: The trained Gradient Boosting pipeline (including the ColumnTransformer preprocessor) will be serialised with joblib. Model versioning will be managed via Git tags so that future re-trained models can be rolled out without breaking existing integrations.

Visualisation Dashboard: A price heatmap and brand-vs-condition comparison chart will be embedded in the Streamlit app to help users understand which vehicle categories command premium prices.

Retraining Pipeline: A GitHub Actions workflow will be scheduled quarterly to re-download the latest Kaggle dataset, re-run the full notebook, evaluate performance, and automatically update the deployed model if the new R^2 exceeds the current threshold.